

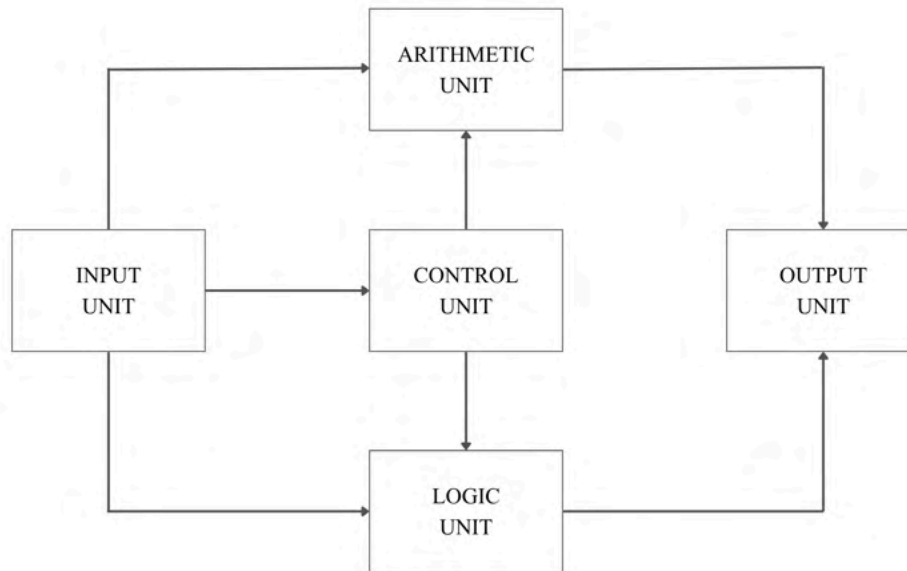
4-BIT ALU DESIGN REPORT

I. INTRODUCTION

- An ALU (Arithmetic Logic Unit) is a fundamental building block of digital systems and CPUs (Central Processor Units). It performs arithmetic and logic operations on binary data
- This project presents the design and implementation of a 4-bit ALU supporting basic arithmetic and logic operations such as addition, subtraction, AND, OR, XOR, NOT using 74HC series ICs

II. DESIGN OVERVIEW

- The design consists of 5 main blocks with the following Block Diagram:



III. LOGIC DESIGN

1. Addition Operation

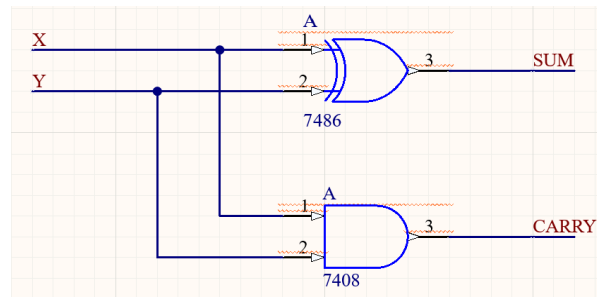
1.1. Half-adder

- Binary addition is performed in the same way as decimal addition except that the values of individual digits can be only 0 or 1. Considering the addition of 2 one-bit numbers, there are four possible combinations defined in the form of a truth table shown in the table below:

x	y	Carry c	Sum s
0	0	0	0
0	1	0	1
1	1	1	0
1	0	0	1

- Two bits are needed to represent the result of the operation:
 - + The right-most bit is called the sum, $s = x \oplus y$
 - + The left-most bit is called the carry, $c = xy$

- The circuit representing these functions is shown in the figure below. This circuit, which implements the addition of two one-bit numbers, is called a *half-adder*



1.2. Full-adder

- Considering the addition of larger numbers with multiple bits, it is necessary to add each pair of bits separately. For each bit position i , the addition operation may include two bits and a carry-in from bit position $i-1$. For bit position 0, carry-in = 0

- This observation leads to the design of a logic circuit that produces the two outputs: result of the sum and carry-in for the next operation

- The truth table and Karnaugh maps for these functions are shown in the figure below:

Carry-in	x	y	Carry-out	Sum
0	0	0	0	0
0	0	1	0	1
0	1	1	1	0
0	1	0	0	1
1	0	0	0	1
1	0	1	1	0
1	1	1	1	1
1	1	0	1	0

a. Truth table

	xy	00	01	11	10
$carry-in$	0		1		1
	1	1		1	

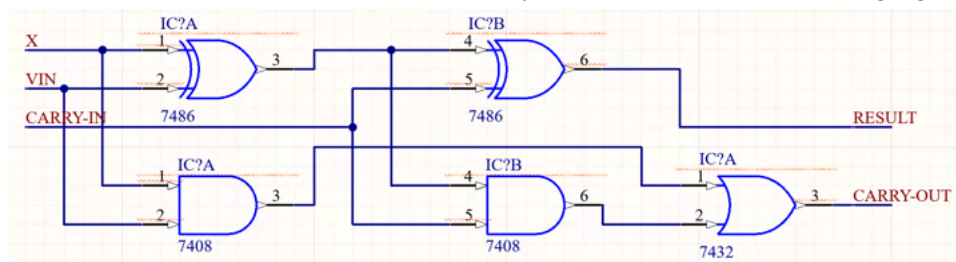
$$s = x \oplus y \oplus cin$$

	xy	00	01	11	10
$carry-in$	0			1	
	1		1	1	1

$$c = xy + cin (x \oplus y)$$

b. Karnaugh maps

- A circuit realization of the functions of result and carry-out is shown in the following figure:



Implementation of the *full-adder* circuit

2. Subtraction Operation

2.1. 2's Complement Representation

- To represent a negative number in binary, we commonly use the 2's complement method. In the 2's complement scheme, a negative number is obtained by subtracting its equivalent positive number from 2^n . Considering a four-bit number as an example: $-5 = 10000 - 0101 = 1011$

- To find a 2's complement of a number, we invert all bits to find its 1's complement and then add 1 to the result.

2.2. Adder and Subtractor Unit

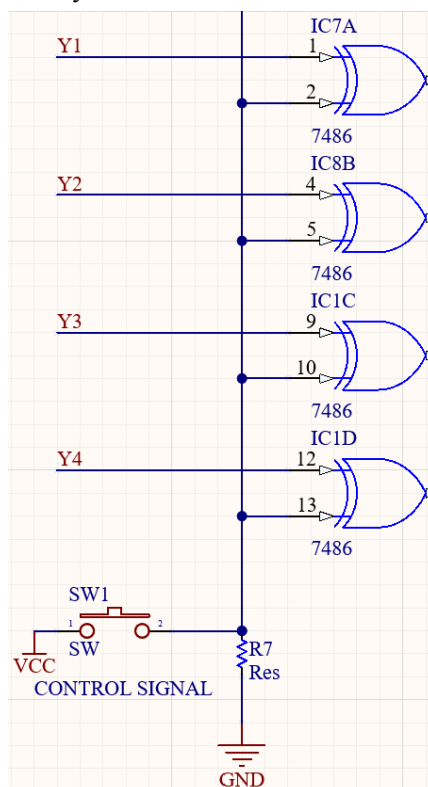
- The only difference between performing addition and subtraction is that for subtraction, it is necessary to use the 2's complement method of one operand: $X - Y = X + (-Y)$

- We know that a 2's complement can be obtained by adding 1 to the 1's complement. Adding 1 in the least-significant bit position can be accomplished by setting the carry-in to 1

- Inverting all bits can be done with NOT gates. But we need a more flexible circuit to combine addition and subtraction operations into one unit. Considering that XOR gates can be used to choose between true and inverted version of an input value, under the control of the other input, this idea can be applied in the design of the arithmetic unit

- Let a control signal value be 0 for addition and 1 for subtraction; and let each bit of Y be connected to one input of an XOR gate with the other input connected to the control signal. The outputs of the XOR gates represent Y when the signal is 0, and represent the inverted version of Y when the signal equals 1

- Note that the control signal is also connected to the first carry-in. This makes the first carry-in = 1 when performing subtraction, thus adding the 1 that is needed to form the 2's complement of Y. When the addition operation is performed, the carry will be 0



XOR gates with control signal for conditional inversion

3. Arithmetic Overflow

- If n bits are used to represent signed numbers, then the result must be in the range -2^{n-1} to $2^{n-1} - 1$. If the result does not fit in this range, then we say that *arithmetic overflow* has occurred. To ensure that the result of an operation displayed is correct, it is necessary to detect the occurrence of overflow

- The key to determine whether overflow occurs is the carry-out from the most-significant bit position, called c_3 , and the sign bit position, called c_4

$$\text{Overflow} = c_3 \oplus c_4$$

IV. IMPLEMENTATION

- Input Interface: 8-bit switches

- Output Interface:

+ red LEDs: result

+ blue LED: carry-out

+ green LED: overflow detection

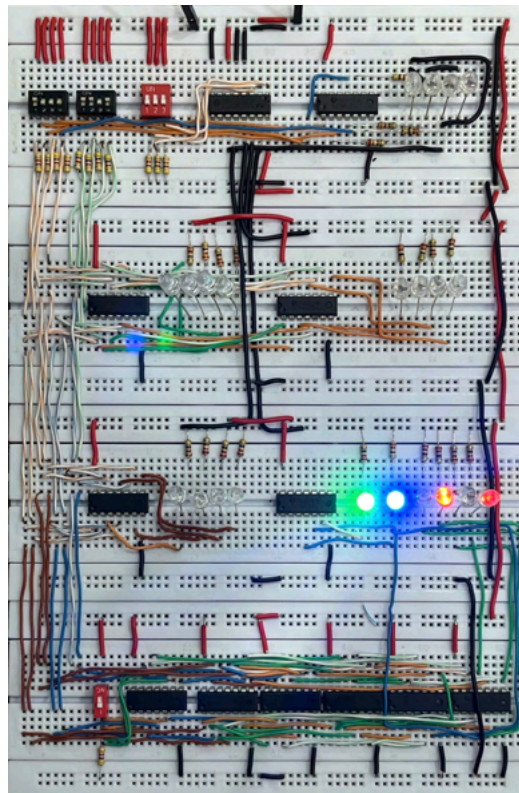
- Control Unit: Decoder IC 74HC138

- Arithmetic and Logic Unit: IC 74HC08 (AND), IC 74HC32 (OR), IC 74HC86 (XOR), IC 74HC04 (NOT)

V. RESULT

1. Arithmetic Unit

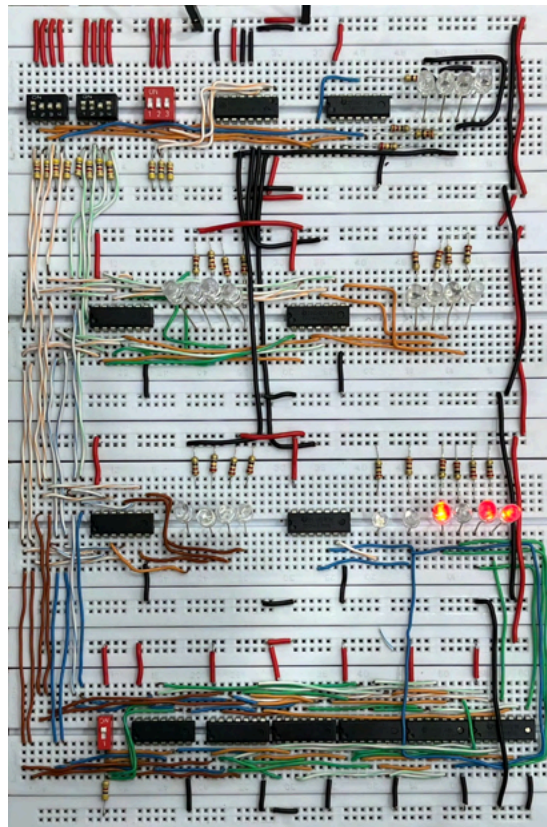
1.1. Addition Operation



$X = 1000$; $Y = 1101$; Result = 0101

Because the LED that represent overflow detection (green LED) is on, the result shown is incorrect

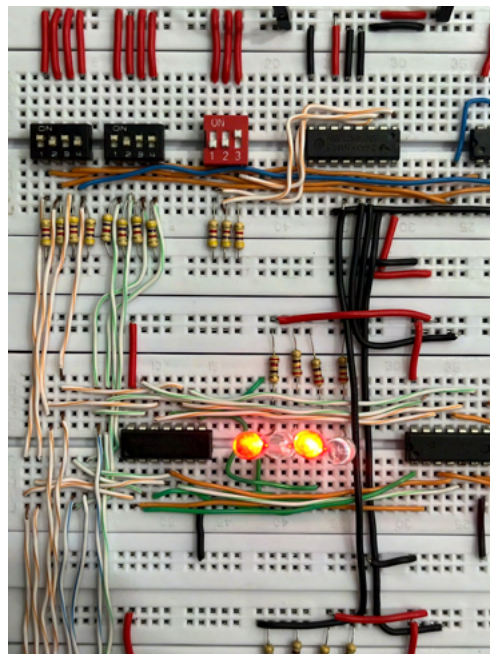
1.2. Subtraction Operation



$X = 1000$; $Y = 1101$; Result = $X - Y = 1011$

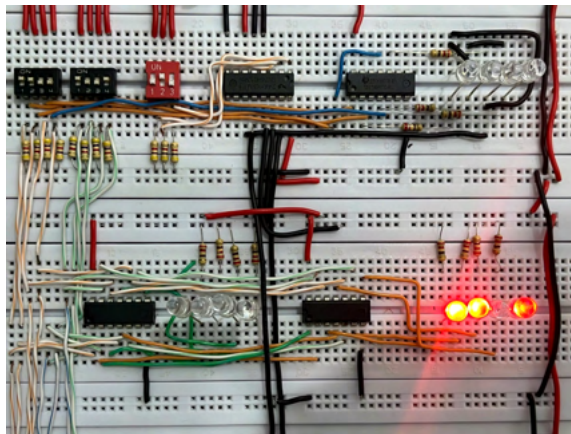
2. Logic Unit

2.1. AND Gates



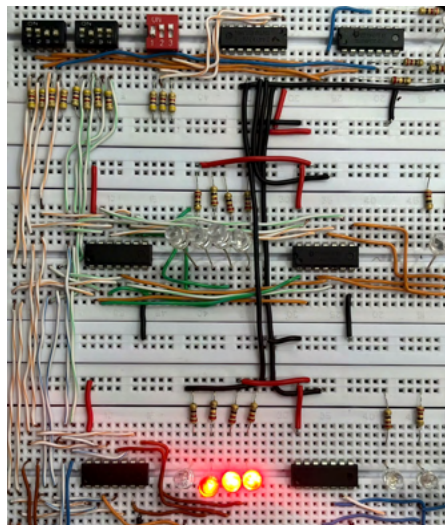
$X = 1010$; $Y = 1011$; Result $X \text{ AND } Y = 1010$

2.2. OR Gates



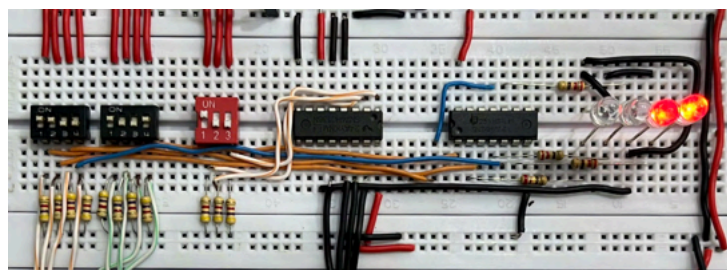
$X = 1100; Y = 1001; \text{Result} = X \text{ OR } Y = 1101$

2.3. XOR Gates



$X = 1100; Y = 1011; \text{Result} = X \text{ XOR } Y = 0111$

2.4. NOT Gates



$X = 1100; \text{Result} = \text{NOT } X = 0011$

VI. REFERENCE

Fundamentals of Digital Logic with Verilog Design (Third Edition) - Stephen Brown and Zvonko Vranesic